

→ How agents decide and when they stop —:

Haiku
opus
sonnet

→ If every time you don't want to call the API through export variable then simply, put it in `.env` file in the project.

Creating a project and then using API key. Using Claude SDK models via Anthropic

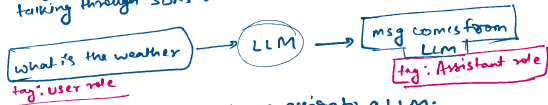
```
PROJECT
├── .env
├── agent.py
└── api_check.py

api_check.py
1 import anthropic
2
3 client = anthropic.Anthropic()
4 # The SDK automatically reads ANTHROPIC_API_KEY from the environment
5 # No need to pass the key manually
6
7 response = client.messages.create(
8     model="claude-haiku-4.5",
9     max_tokens=100,
10    messages=[
11        {"role": "user", "content": "Say hello in one word."}
12    ]
13)
14
15 print(response.content[0].text) # Should print: Hello
```

→ Roles in Claude. { Roles define what a particular entity does. }

- ① System role
- ② User role → `US` In Claude, we can assign role
- ③ Assistant Role → Can be Claude etc.

→ We don't explicitly mention role in Claude while prompting. Whenever we msg to Claude, our messages gets tagged under "User role". But as we are talking through SDKs we have to be more technical.

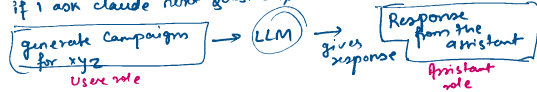


→ System role: whatever role we assign to a LLM.

eg: you are a medical expert

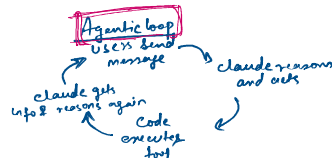
we just define it once.
LLM arranges that role and acts like it.

Now if I ask Claude next questions.



chat vs Agent

Standard chat
User sends msg
↓
Claude responds
↓
Done.



Agentic loop

① Send Req.
call API with messages (array+tool)

② Inspect
stop_reason
check if tool use or
end_turn

④ Append
Add results to
history loop to stop.

③ Execute
Run local tool
if tool use

This loop continues unless we've got a satisfied response from tool usage.
Roles and message arrays are imp. Because Claude is 100% Stateless meaning we have to send role & complete msg history.

- ③ Messages array →
→ Whenever an agent encounters a "stop reason", it encounters whether to go for tool usage or if the job is done and that is an end turn flag.
④ During this back & forth conversation, messages that are exchanged → Being stateless. Claude will forget conversation next time you trigger the API.
So, every time we need to attach entire message history.

We learned that an agent will pause its execution for a stop reason.
stop reason is of 2 types → tool use (continues this cycle until reaches end turn) and end_turn

→ While using Claude API, we need to explicitly state the role
eg: { "role": "user", "content": " ", "message": " " }

→ During the back & forth of Claude tool use & end turn, messages are exchanged. But in API, Claude forgets message if we open it next time. Hence we include entire message history everytime we use the API. Same logic for Agent.

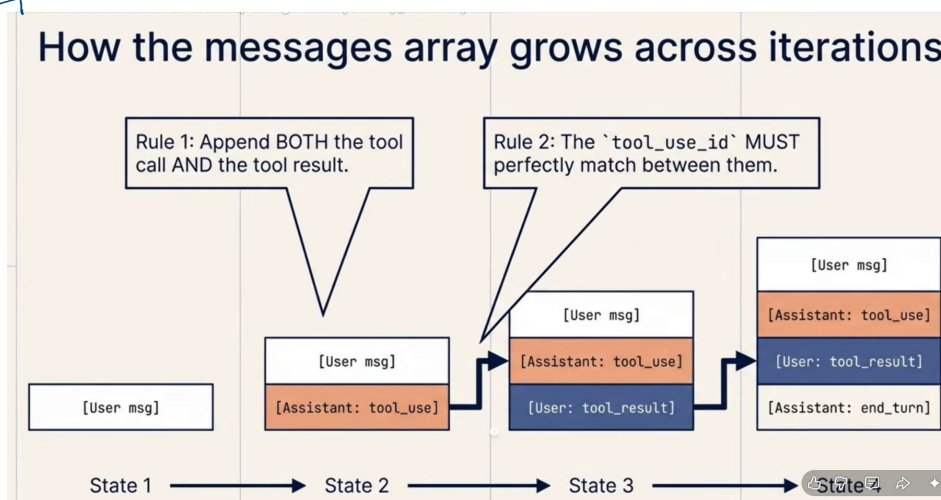
Any Python function that we create, Claude cannot call it automatically.
It requests a **tool call**. Our code runs it & results are returned. <33:00>
→ def run_agent(user_message: str) → str: { run the agent till we find a final answer. }
... "user message" ← user message.

Any python function that we create, ~
 # requests a tool call. Our code runs it & results are returned. <33:00>

```
def run_agent(user_message: str) -> str:
    message = [{"role": "user", "content": user_message}]
    # Safety valve
    MAX_ITERATION = 50
    iteration = 0
    while iteration < MAX_ITERATION:
        iteration += 1
        # Claude cannot call the function tool automatically. we need to send the
        # tool to Claude API. Claude then returns a response with "stop_reason"
        response = client.messages.create(
            model="claude-haiku-4.5",
            max_tokens=4096,
            tools=tools,
            message=message
        )
        # Exit condition. If we've defined stop_reason and turn some text.
        if response.stop_reason == "end_turn":
            for block in response.content:
                if block.type == "text":
                    return block.text
            return ""
        elif response.stop_reason == "tool_use":
            message.append({
                "role": "assistant",
                "content": response.content
            })
            # meaning Claude works to call a tool.
            # we append assistant message.
            # since Claude API is stateless
            # we need to provide entire
            # msg history.
```

```
main()
user_msg = str("Where is my order 1234")
run_agent(user_msg)
```

How the messages array grows across iterations



.. case. model drove the tool selection.
 in .. Claude judged the



on our case, model drove the tool selection.
 It was agentic meaning - (model driven) agentic. Claude judged the
stop reason. It decided that what tool to call next.
 we cannot hardcode rigidly saying fixed sequence of tool execution.
 on agentic execution, Claude can call tool in any order.

Why the model drives tool selection

Exam Target: Task 1.1

Model-Driven (Agentic)	Pre-Configured (Not Agentic)
• Claude reads the full conversation context. • Decides which tool to call next based on reasoning. • Can call tools in any order, skip unnecessary ones. • Intelligence lives in the model.	• Hardcoded if/then conditional logic. • Rigid, fixed sequence of tool execution. • Fragile when novel situations or requirements change. • Intelligence lives in the code.

3 Anti patterns on the text -

pattern can be regex ek. basis which we match.
 => Here we go for anti-pattern
Anti-pattern 1 -> always use stop_reason to terminate the loop.

Anti-pattern 1 — parsing Claude's words

Wrong Way	Right Way
<pre>if "I have completed" in response.text: break</pre>	<pre>if stop_reason == "end_turn": break</pre>
Why it fails: Claude's wording varies. On a tool_use turn, there might be NO text block, causing a fatal IndexError crash.	Takeaway: Do not read natural language. Use the structured, mechanical API signal.

Other 2 are listed below -

Three anti-patterns tested on the exam

Root Cause: Substituting a hacky heuristic for the structured API signal.

Natural Language Parsing	Iteration Caps as Primary Stops	Content Type Checking
Trying to read the AI's mind by parsing words instead of checking stop_reason.	Using a maximum loop counter to end the task instead of a safety valve.	Breaking the loop when encountering text, ignoring mixed content blocks.

Anti-pattern 2 — iteration caps as primary stops

Wrong Way

```
if iterations >= 5:
    break
```



Why it fails: Arbitrary caps ignore reality. Tasks finishing in 3 iterations waste calls. Tasks needing 8 iterations are prematurely cut off.

Right Way



Takeaway: Use an iteration cap as a safety valve / circuit breaker, but NEVER as the primary termination condition.

Anti-pattern 3 — content type checking

Wrong Way

```
if response.content[0].type == "text":
    break
```



Why it fails: Claude frequently thinks out loud ("Let me look that up" + tool_use). Breaking on text silently drops the pending tool call. The agent fails without throwing an error.

Right Way

Takeaway: Never substitute content type inspection for stop_reason. They measure entirely different things.

The complete agentic loop — reference implementation

```
while iteration < MAX_ITERATIONS:
    response = client.messages.create(...)

    if response.stop_reason == "end_turn":
        break

    messages.append({"role": "assistant", "content": response.content})

    # ... execute tool based on response ...

    messages.append({"role": "user", "content": [
        {"type": "tool_result", "tool_use_id": block.id, "content": result}
    ]})
```

The Safety Valve

The Primary Exit

Append Step 1:
Save Assistant
Tool Call

Append Step 2:
Save User Result
(Matching ID)

Synthesis: The Architect's Cheat Sheet

The 5 Golden Rules

1. `end_turn` is the ONLY valid loop exit.
2. `tool_use` means execute and continue.
3. Always append assistant first, then user.
4. `tool_use_id` must perfectly match.
5. The API is stateless; send the full history.

The 3 Anti-Patterns to Spot on Exams

Parsing words → Causes crashes.	Iteration cap → Cuts off early.	Text content check → Drops tools silently.
------------------------------------	------------------------------------	---

How the exam tests this: Look for the missing history append, the mismatched ID, or the hacked stop condition.

First call is
anyway user. But
post that it has to be
assistant → user → Assistant